

uc3m-thesis-ieee

Package Documentation — v0.4.0

Universidad Carlos III de Madrid thesis template, IEEE style

1. Configuration

The template exports a single `conf` function, which you apply to your document with `#show: conf.with(...)`.

1.1. Minimal example

codly

```
1  #import "@preview/uc3m-thesis-ieee:0.4.0": conf
2
3  #show: conf.with(
4    title: "My Bachelor Thesis",
5    author: "GUL UC3M",
6    degree: "Computer Science and Engineering",
7    advisors: ("Prof. L",),
8    location: "Leganés, Madrid",
9    thesis-type: "TFG",
10    date: datetime(year: 2025, month: 6, day: 15),
11    language: "en",
12    abstract: (
13      body: [A short description of my thesis.],
14      keywords: ("Keyword 1", "Keyword 2", "Keyword 3"),
15    ),
16    genai-declaration: (usage: false),
17  )
18
19 = Introduction
20
21 My thesis starts here.
```

1.2. All parameters

1.3. lib

- `conf()`
- `outline-entry-formatter()`

1.3.1. conf

Main configuration function.

Recommended to use with `#show: conf.with(...)`.

1.3.1.1. Parameters

```

conf(
  doc: content,
  title: str,
  author: str,
  degree: str,
  advisors: array,
  location: str,
  thesis-type: str,
  date: datetime,
  bibliography-content: content,
  language: str,
  style: str,
  titlepage-style: str auto,
  table-style: str auto,
  figure-style: str auto,
  figure-spacing: length none,
  double-sided: bool,
  logo: str,
  short-title: str,
  date-format: str auto,
  license: bool,
  flyleaf: bool,
  epigraph: dictionary none,
  abstract: dictionary,
  english-abstract: dictionary,
  acknowledgements: content none,
  outlines: dictionary none,
  appendixes: content none,
  glossary: array content none,
  abbreviations: dictionary content none,
  genai-declaration: dictionary content
) -> content

```

1.3.1.1.1. doc content

Thesis contents.

1.3.1.1.2. title str

Title of the thesis.

Default: none

1.3.1.1.3. author `str`

Author name.

Default: `none`

1.3.1.1.4. degree `str`

Degree name (e.g. "Computer Science and Engineering").

Default: `none`

1.3.1.1.5. advisors `array`

List of advisor names.

Default: `none`

1.3.1.1.6. location `str`

Presentation location.

Default: `none`

1.3.1.1.7. thesis-type `str`

Type of thesis ("TFG" or "TFM").

Default: `none`

1.3.1.1.8. date `datetime`

Presentation date.

Default: `none`

1.3.1.1.9. bibliography-content `content`

Bibliography contents, usually calling bibliography.

Default: `none`

1.3.1.1.10. language `str`

"en" or "es".

Default: `none`

1.3.1.1.11. style `str`

Visual style, mainly affecting headings, headers, and footers. The available styles are `strict`, which strictly follow's the university library's guidelines, `clean`, based on clean-dhbw, and `fancy`, based on my original LaTeX version.

Default: `"fancy"`

1.3.1.1.12. titlepage-style `str` or `auto`

Style for the titlepage (see `style`). If set to `auto`, uses the main style.

Default: `auto`

1.3.1.1.13. table-style `str` or `auto`

Style for the table caption, either `"clean"` or `"ieee"`. If set to `auto`, uses the default for the main style (`"clean"` for `clean` style, `"ieee"` for the rest).

Default: `auto`

1.3.1.1.14. figure-style `str` or `auto`

Style for the figure caption, either `"clean"` or `"ieee"`. If set to `auto`, uses the default for the main style (`"clean"` for `clean` style, `"ieee"` for the rest).

Default: `auto`

1.3.1.1.15. figure-spacing `length` or `none`

Extra spacing to give to figures and tables. If `none`, no extra spacing.

Default: `0.75em`

1.3.1.1.16. double-sided `bool`

Whether to use double-sided pages. This is not allowed in the `strict` style.

Default: `false`

1.3.1.1.17. logo `str`

Type of logo (`"old"` or `"new"`).

Default: `"new"`

1.3.1.1.18. short-title str

Shorter version of the title, to be displayed in the headers. Only applies if double-sided is set to true.

Default: none

1.3.1.1.19. date-format str or auto

Date format. Use auto or specify the format using the [Typst format syntax](https://typst.app/docs/reference/foundations/datetime/#format).

Default: auto

1.3.1.1.20. license bool

Whether to include the CC BY-NC-ND 4.0 license.

Default: true

1.3.1.1.21. flyleaf bool

Whether to include a blank page after the cover.

Default: true

1.3.1.1.22. epigraph dictionary or none

A short quote that guided you through the writing of the thesis, your degree, or your life. Consists of quote (of type content), the body or text itself, author (of type str), the author of the quote and, optionally, source (of type str), where the quote was found.

Default: none

1.3.1.1.23. abstract dictionary

A short and precise representation of the thesis content. Consists of body (of type content), the main text, and keywords, an array of key terms (of type str) (see [IEEE Taxonomy](https://www.ieee.org/content/dam/ieee-org/ieee/web/org/pubs/ieee-taxonomy.pdf)).

Default: none

1.3.1.1.24. english-abstract dictionary

An english translation of the abstract. Compulsory for spanish works, invalid for english ones.

Default: none

1.3.1.1.25. acknowledgements `content` or `none`

Text where you give thanks to everyone that helped you.

Default: `none`

1.3.1.1.26. outlines `dictionary` or `none`

Set of extra outlines to include (figures, tables, listings), and extra custom outlines (custom, an array of contents – typically the result of calling `outline`).

Default: `none`

1.3.1.1.27. appendixes `content` or `none`

Set of appendixes.

Default: `none`

1.3.1.1.28. glossary `array` or `content` or `none`

Glossary entries. If `array` is provided, it will use the `glossarium` library. If `content` is passed, it will display that content, without applying any styling.

Default: `none`

1.3.1.1.29. abbreviations `dictionary` or `content` or `none`

Abbreviations, acronyms and initials used throughout the thesis. You can provide a map (dictionary of strings) or a custom one (`content`).

Default: `none`

1.3.1.1.30. genai-declaration `dictionary` or `content`

Information about the use of Generative AI in the thesis. You can supply your own content, or use the university's template, by supplying a dictionary. See the example for more details.

Default: `none`

1.3.2. outline-entry-formatter

Formats an outline entry.

1.3.2.1. Parameters

```
outline-entry-formatter(
  it: content,
  prefix: auto content,
  body: auto content,
  fill: boolean,
  page: boolean,
  above: auto fraction relative,
  spacing: auto fraction relative,
  text-size: auto length,
  text-weight: int str,
  color: color,
  indented: boolean,
  justified: boolean
) -> content
```

1.3.2.1.1. it content

Outline entry.

1.3.2.1.2. prefix auto or content

Entry prefix (e.g. [Chapter 1]). If auto, default prefix.

Default: auto

1.3.2.1.3. body auto or content

Entry body (e.g. [My chapter]). If auto, default body.

Default: auto

1.3.2.1.4. fill boolean

When true, add dots between the body and the page number.

Default: true

1.3.2.1.5. page boolean

When true, shows the page number.

Default: true

1.3.2.1.6. above auto or fraction or relative

The spacing between this block and its predecessor (block.above).

Default: auto

1.3.2.1.7. spacing `auto` or `fraction` or `relative`

The spacing around the block (`block.spacing`). When `auto`, inherits the paragraph spacing.

Default: `auto`

1.3.2.1.8. text-size `auto` or `length`

Text size. When `auto`, default text size. (`text.size`)

Default: `auto`

1.3.2.1.9. text-weight `int` or `str`

Text weight (`text.weight`).

Default: `"regular"`

1.3.2.1.10. color `color`

Entry text color. Includes link color.

Default: `black`

1.3.2.1.11. indented `boolean`

Whether to indent the entry.

Default: `true`

1.3.2.1.12. justified `boolean`

When `false`, wraps the entry in a block to prevent justification.

Default: `true`

1.4. generative-ai

- `genai-template()`

1.4.1. genai-template

UC3M template for declaration of use of AI in the project.

1.4.1.1. Parameters

```
genai-template(
  language: str,
  usage: bool,
  data-usage: dictionary,
  technical-usage: dictionary,
  usage-reflection: content
)
```


1.4.1.1.1. language `str`

The language of the project (e.g. “en”, “es”).

1.4.1.1.2. usage `bool`

Whether AI is used in the project.

1.4.1.1.3. data-usage `dictionary`

AI data usage declaration.

1.4.1.1.4. technical-usage `dictionary`

Technical usage of AI.

1.4.1.1.5. usage-reflection `content`

Reflection on the usage of AI.

1.5. Abstract

The abstract parameter takes a dictionary with the following keys:

Key	Type	Description
body*	content	The abstract text.
keywords*	array of str	Between 2 and 5 keywords. See IEEE Taxonomy.

```

1  abstract: (
2    body: [
3      This thesis presents an implementation of interrupts and timers
4      in the CREATOR simulator.
5    ],
6    keywords: ("Operating Systems", "Interrupts", "RISC-V", "Simulation"),
7  ),

```

For Spanish documents, it is required to provide an english-abstract with the same structure:

```

1  language: "es",
2  abstract: (
3    body: [Una breve descripción de mi tesis.],
4    keywords: ("Sistemas Operativos", "Interrupciones", "RISC-V"),
5  ),
6  english-abstract: (
7    body: [A brief description of my thesis.],
8    keywords: ("Operating Systems", "Interrupts", "RISC-V"),
9  ),

```

1.6. Epigraph

An optional opening quote placed before the abstract. Provide a dictionary with:

Key	Type	Description
quote*	content	The quote text.
author*	str	The author of the quote.
source	str	(Optional) The source where the quote was found.

```

1 epigraph: (
2     quote: [The purpose of abstracting is not to be vague, but to create a new
3     semantic level in which one can be absolutely precise.],
4     author: "Edsger W. Dijkstra",
5     source: "The Humble Programmer, 1972",
6 ),

```

typ

1.7. Outlines

The outlines parameter accepts an optional dictionary with the following boolean keys. All keys are optional (default false):

Key	Type	Description
figures	bool	Include a list of figures.
tables	bool	Include a list of tables.
listings	bool	Include a list of code listings.
custom	array of content	Additional custom outlines (e.g. algorithms).

```

1 outlines: (
2     figures: true,
3     tables: true,
4     listings: false,
5     // custom: (
6     //     outline(
7     //         title: [List of Algorithms],
8     //         target: figure.where(kind: "algorithm"),
9     //     ),
10    // ),
11 ),

```

typ

1.8. Abbreviations

You can pass abbreviations as a dictionary mapping abbreviations to their expansions (automatically sorted alphabetically):

```

1 abbreviations: (
2     API: "Application Programming Interface",
3     CPU: "Central Processing Unit",
4     ML: "Machine Learning",
5     TFG: "Trabajo de Fin de Grado",

```

typ

```
6 ),
```

Or as custom content if you need full control over the layout:

```
1 abbreviations: [typ
2   #table(
3     columns: (auto, lfr),
4     [*API*], [Application Programming Interface],
5     [*ML*], [Machine Learning],
6   )
7 ],
```

1.9. Glossary

The template integrates with [glossarium](https://typst.app/universe/package/glossarium/) for automatic glossary and acronym management.

Define your entries in a separate file (e.g. config/glossary.typ):

```
1 #let glossary-entries = (typ
2   (
3     key: "API",
4     short: "API",
5     long: "Application Programming Interface",
6     description: "A set of protocols and tools for building software.",
7   ),
8   (
9     key: "ML",
10    short: "ML",
11    long: "Machine Learning",
12    description: "A subset of AI that enables computers to learn from
13      data.",
14  ),
15 )
```

Then import it in your main file and pass it to conf:

```
1 #import "config/glossary.typ": glossary-entriestyp
2
3 #show: conf.with(
4   // ...
5   glossary: glossary-entries,
6 )
```

To reference a glossary term in the body text, use #gls("key") (singular) or #glspl("key") (plural). These are re-exported from glossarium and available after the #show: conf.with(...) call:

```
1 #import "@preview/uc3m-thesis-ieee:0.4.0": conftyp
2 #import "@preview/glossarium:0.5.9": gls, glspl
```

```

3
4 // In the body:
5 The #gls("API") exposes several endpoints.
6 Multiple #glspl("ML") models were evaluated.

```

Alternatively, you can pass fully custom content to glossary if you don't want to use glossarium.

1.10. Generative AI Declaration

The `genai-declaration` parameter is **required** by the university. You can either:

1.10.1. Option A: Use the built-in template (recommended)

If you did not use generative AI (`usage: false`):

```
1  genai-declaration: (usage: false), typ
```

If you did use generative AI (`usage: true`), it is required to provide the full declaration:

Pass a dictionary with the following keys:

Key	Type	Required	Description
<code>usage*</code>	<code>bool</code>	Always	Whether you used generative AI in the thesis.
<code>data-usage</code>	<code>dictionary</code>	If <code>usage</code> is true	Data usage declaration. See below.
<code>technical-usage</code>	<code>dictionary</code>	If <code>usage</code> is true	Technical usage declaration. See below.
<code>usage-reflection</code>	<code>content</code>	If <code>usage</code> is true	Personal reflection on AI usage.

data-usage keys — all are `bool` (`true` = YES, `false` = NO):

Key	Description
<code>confidential</code>	Did you submit confidential data (trade secrets, proprietary info)?
<code>copyright</code>	Did you submit copyrighted materials?
<code>personal</code>	Did you submit personal data?
<code>tos</code>	Did your usage respect the tool's terms of use?

technical-usage keys — include only those that apply, with a brief description of the prompt and interaction. Items are automatically grouped into two sections matching the official form:

Documentation and drafting: reflection, review, research, references, summary, translation

Develop specific content: assistance-coding, generating-content, optimization, data-processing, idea-inspiration, other

An optional `tool` key (`str`) declares the AI system name and version (e.g. "ChatGPT 4o").

```

1  genai-declaration: ( typ
2    usage: true,
3    data-usage: (
4      // true = YES / used with authorization; false = NO / not used

```

```

5     confidential: false,
6     copyright: false,
7     personal: false,
8     tos: true,
9   ),
10  technical-usage: (
11    // Name of the AI system and version (optional):
12    // tool: "ChatGPT 4o",
13
14    // — Documentation and drafting —————
15    reflection: [I asked for a list of alternatives to address the
16      problem stated at the beginning of the paper.],
17    // review: [...],
18    // research: [...],
19    // references: [...],
20    // summary: [...],
21    // translation: [...],
22
23    // — Develop specific content —————
24    assistance-coding: [I asked for explanations about a Python code
25      snippet and for help correcting an error.],
26    // generating-content: [...],
27    // optimization: [...],
28    // data-processing: [...],
29    // idea-inspiration: [...],
30    // other: [...],
31  ),
32  usage-reflection: [
33    Overall, I found generative AI useful for boilerplate tasks and
34    quick explanations, but it required careful verification of all
35    generated content. It did not replace critical thinking or
36    domain-specific knowledge.
37  ],
38 ),

```

1.10.2. Option B: Provide custom content

If you need full control, pass any content directly:

```

1  genai-declaration: [
2    // Your custom AI declaration content here.
3  ],

```

typ

1.11. Bibliography

Pass the result of Typst's built-in `bibliography()` call:

```

1  // config/bibliography.typ
2  #let bibliography-file = "../references.bib"

```

typ

```

3 #let bibliography-style = "ieee"
4 #let bibliography = bibliography(bibliography-file, style: bibliography-
  style)

```

```

1 // report.typ
2 #import "config/bibliography.typ": bibliography
3
4 #show: conf.with(
5   // ...
6   bibliography-content: bibliography,
7 )

```

typ

2. Styles

The template offers three visual styles, controlled by the style parameter.

2.1. "fancy" (default)

An opinionated style inspired by the original [LaTeX template](<https://github.com/ldcas-uc3m/thesis-template>), with:

- Decorative chapter title pages with a horizontal rule and chapter number/name centered
- Serif headers showing the current chapter name and page number
- IEEE-style figure captions and table captions
- **Libertinus Serif** body font

2.2. "clean"

A clean, minimal style inspired by [clean-dhbw](<https://github.com/roland-KA/clean-dhbw-typst-template>), with:

- Large chapter number in the background on chapter pages
- Simple running header showing the chapter name
- Clean-style figure and table captions
- **Libertinus Serif** body font

2.3. "strict"

Strictly follows the [UC3M library guidelines](<https://uc3m.libguides.com/en/TFG/writing>):

- Times New Roman font, 12pt
- Centered uppercase chapter headings
- Page numbers in the footer (Roman numerals in front matter, Arabic in body)
- No decorative elements
- Double-sided layout is **not** permitted with this style

> [!NOTE] > You can mix styles for the title page and body independently using `titlepage-style`. For example, to use a fancy title page with a strict body: `style: "strict", titlepage-style: "fancy"`.